

CLASSIFICAZIONE AUTOMATICA DI NUMERI

Gruppo di lavoro

- Michele_Loconte, matricola 741221,
m.loconte27@studenti.uniba.it

AA 2025-2026

GITHUB: <https://github.com/Imik00ll/ICON.git>

Introduzione

Il progetto ha come obiettivo la classificazione automatica delle immagini di numeri scritti a mano, in una griglia di 8x8 pixel.(per un totale di 64 feature)

Rientra nell' apprendimento automatico dove vengono analizzati e confrontati i 3 modelli usati: Decision Tree , Support Vector Machine, K-Nearest Neighbors.

Sommario

E' un sistema di apprendimento automatico supervisionato che apprende dal dataset "load_digits" incluso nella libreria scikit-learn che contiene circa 1797 sample di numeri scritti a mano .

Quindi in fase di addestramento ogni matrice 8x8 subisce un appiattimento (flattening) lineare che la trasforma in un vettore unidimensionale a 64 feature continue:

```
dataset = load_digits()
X = dataset['data']    # Matrice di forma (1797, 64)
y = dataset['target']  # Vettore target con classi da 0 a 9
```

Ogni feature corrisponde al valore di intensità di grigio del rispettivo pixel che va da [0,16] nell'intervallo.

ARGOMENTI DI INTERESSE.

Costruzione del ground truth (Preprocessing e Raccolta):

Poiché all'inizio non dispongo di una classificazione manuale esterna per ogni singola immagine, ho usato un dataset validato (load_digits di scikit-learn).

- **1.** Sono partito dal preparare e “pulire” dati (Data Cleaning) e della gestione delle feature del dataset standard di scikit-learn (load_digits), strutturando le matrici bitmap originali di 8x8 pixel in array unidimensionali a 64 feature continue e associandole ai target discreti (da 0 a 9).
- **1.2 Raccolta dei dati in un dataset rappresentante il ground truth:** organizzazione strutturata delle 1797 istanze complessive suddivise in feature d'ingresso (x) e classi obiettivo (y).

Riproduzione del ground truth (Spazio di ricerca, Grafi e Apprendimento supervisionato):

- **2.1 Grafi e Spazio di Ricerca:** Il problema è stato modellato come una ricerca nello spazio degli stati definito dalla struttura ad albero generata induttivamente dal Decision Tree. In questo caso, ogni nodo dell'albero rappresenta uno stato, la radice funge da stato iniziale e le foglie costituiscono gli stati obiettivo (goal). Gli archi logici indicano le azioni (i test decisionali sui valori dei pixel). L'algoritmo di ricerca euristica A* esplora tale spazio discreto per individuare il cammino ottimo verso la foglia associata alla classe target, calcolando il costo del cammino $g(n)$ come profondità di navigazione nell'albero e adottando come euristica $h(n)$ l'indice di impurità di Gini, la cui minimizzazione stima la vicinanza a un nodo omogeneo (puro).
-
- **2.2 Apprendimento Supervisionato:** Progettazione, addestramento, ottimizzazione degli iperparametri (tramite GridSearchCV) e ho scelto una validazione rigorosa (tramite Nested 5-Fold Cross-Validation) dei tre modelli analizzati:
 - **Albero di Decisione:** Rappresenta un modello di apprendimento induttivo e simbolico di tipo "eager learning", che generalizza la conoscenza a priori durante l'addestramento. La sua architettura

gerarchica permette di tradurre la partizione dello spazio delle feature direttamente in una Knowledge Base esplicita: ogni ramo radice-foglia costituisce una regola di produzione dichiarativa (IF-THEN) formulabile in logica proposizionale, garantendo un modello White-Box e interpretabile.

-
- **Support Vector Machine (SVC)** con kernel linear. Rappresenta un paradigma di apprendimento sub-simbolico e geometrico basato sulla teoria statistica dell'apprendimento e sull'ottimizzazione convessa. A differenza dell'Albero, non sintetizza regole dichiarative per la KB: la conoscenza appresa è codificata implicitamente nei pesi w e nel termine di bias di un iperpiano a margine massimo nello spazio vettoriale $\mathbf{R}^{64}$. La decisione avviene per via algebrica individuando i soli vettori di supporto critici, rendendo la rappresentazione sparsa ed efficiente contro l'overfitting.
-
- **K-Nearest Neighbors (KNN)**. algoritmo implementa il paradigma del Case-Based Reasoning (CBR) mediante "lazy learning": non sintetizza un modello generale a priori (come l'Albero di Decisione), ma differisce la generalizzazione alla fase online, valutando la similarità metrica della nuova istanza direttamente contro il repository di casi pregressi noti.

3. Knowledge Base e Rappresentazione della Conoscenza

Una *knowledge base* (KB) è un sistema strutturato che rappresenta fatti sul mondo, consentendo di effettuare ragionamenti su di essi e di dedurre nuove informazioni o risposte. Nel contesto dei sistemi intelligenti, la KB costituisce l'ambiente volto a facilitare la raccolta, l'organizzazione e la distribuzione della conoscenza. Nel caso di studio, la Knowledge Base viene utilizzata per consentire al sistema di apprendere e all'utente di interrogare il dominio di riferimento tramite la classificazione automatica.

3.1 Gestione della Knowledge Base nel Progetto

il progetto implementa una vera e propria Knowledge Base simbolica (esplicita e dichiarativa) estraendo la conoscenza appresa induttivamente dall'Albero di Decisione. A differenza dei modelli sub-simbolici e geometrici come SVM e KNN (che non generano regole logiche esplicite ma operano mediante iperpiani e distanze continue), l'albero di decisione è stato sottoposto a una visita sistematica in profondità (DFS). Tale operazione ha permesso di astrarre e compilare una KB formata da un vocabolario di regole di produzione formali IF-THEN (salvate permanentemente nella directory /kb). In fase di predizione, un motore di inferenza dedicato (Pattern Matching / Forward Chaining) valuta i fatti osservati a run-time contro gli antecedenti di tali clausole per derivare la classificazione finale deduttiva, garantendo un'architettura rigorosamente White-Box e interpretabile.

3.1.1 Fatti (Istanziamento dei Dati)

I fatti rappresentano gli assiomi empirici di partenza presenti nella Knowledge Base, costituiti dai campioni campionati dal dominio reale. Nel sistema, ogni istanza rappresenta una cifra scritta a mano digitalizzata e si compone di relazioni strutturate tra l'immagine e le sue proprietà vettoriali:

1. **digit_features(sample_id, feature_vector)** Rappresenta la relazione strutturale tra un'istanza numerica e le sue 64 caratteristiche di input (i pixel della griglia 8x8 normalizzati e appiattiti). A livello logico, l'istanza presentata al sistema non è un semplice array, ma viene codificata in una TBox/ABox sotto forma di fatti assiomatici atomici ed esportati su file (es. `fatto(pixel\_36, 12.0)`, `fatto(pixel\_42, 0.5)` e `ground\_truth\_target(4)`). Su questo insieme di fatti si attivano le regole inferenziali della KB per giungere alla deduzione.
2. **digit_target(sample_id, class_label)** Rappresenta la relazione logica di classificazione tra un'istanza e la sua classe target di appartenenza.
Esempio: `y = dataset['target']` (valore discreto compreso tra 0 e 9).

3.1.2 Valutazione della Complessità e Struttura della Knowledge Base

Al fine di valutare la consistenza e l'espressività della base di conoscenza sintetizzata induttivamente dall'Albero di Decisione, è stata condotta un'analisi quantitativa della sua complessità strutturale:

- **Cardinalità delle Regole:** L'attraversamento in profondità (DFS) dell'albero potato ottimale ($\text{max_depth}=20$, $\text{min_samples_split}=2$) ha generato una Knowledge Base composta esattamente da 168 regole di produzione disgiunte. Ciascuna regola individua un cammino univoco radice-foglia e codifica una regione convessa e deterministica dello spazio delle caratteristiche a 64 dimensioni.
- **Complessità degli Assiomi** (Lunghezza delle Premesse): Ogni regola di produzione presenta un antecedente formato da una congiunzione logica di test relazionali sui singoli pixel. La profondità media delle regole si attesta su 8.4 vincoli per antecedente, con un valore massimo pari a 14 livelli logici di controllo, ampiamente al di sotto del vincolo teorico massimo imposto a priori ($\text{max_depth}=20$). Questo attesta che l'algoritmo non necessita di ramificazioni eccessivamente profonde per discriminare i tratti grafici delle cifre, limitando il bias di rappresentazione ed evitando l'overfitting.
- **Distribuzione delle Regole per Classe:** Le 168 regole estratte coprono l'intero dominio delle 10 cifre decimali (da 0 a 9), manifestando una cardinalità differenziata per ciascun target (con un minimo di 12 regole per cifre calligraficamente uniformi come lo '0' e un massimo di 21 regole per cifre soggette a forte variabilità di tratto come il '3' e l'8'). Ciascuna regola rappresenta una specifica variante allografica (es. '7' con o senza tratto mediano, '4' aperto o chiuso).
 - **Uso e Ciclo di Vita della KB:** L'architettura realizza una netta separazione tra: Fase di Acquisizione della Conoscenza (Offline): Compilazione ed esportazione persistente del vocabolario assiomatico nella directory /kb (regole_n_{target}.txt).
 - **Fase di Inferenza e Consultazione** (Online/Run-time): Presentato un nuovo campione sotto forma di fatti ground (fatti_osservati.txt), il motore deduttivo inferenza_kb esegue un ciclo di pattern matching deterministico determinando la regola attiva in tempo lineare $O(R \times L)$ (dove R è il numero di regole ed L la lunghezza media delle premesse), derivando formalmente la cifra senza ricorrere a euristiche di approssimazione.

3.2 Estensione Ontologica Formale: Paradigma Ibrido Neuro-Simbolico (ML + OntoBK)

Per completare la Knowledge Base a regole proposizionali derivata dall'Albero di Decisione ho integrato un secondo livello di validazione formale basato su un'ontologia espressa in OWL 2 (Description Logics) implementata nel modulo `estensione_OWL.py`. Tale estensione realizza il paradigma ML + OntoBK, fondendo l'apprendimento statistico-induttivo con la conoscenza di fondo (Background Knowledge) formale e deduttiva.

3.2.1 Architettura della TBox (Terminological Box)

La Background Knowledge generale del dominio delle cifre è modellata in modo invariante rispetto alle singole esecuzioni e definisce le seguenti entità:

Classi: La classe madre *Cifra* è suddivisa nelle sottoclassi disgiunte *CifraConAnelli* e *CifraTrattoSemplice*, da cui sussumono le classi specializzate (*CifraZero*, *CifraUno*, *CifraQuattro*, *CifraSei*, *CifraOtto*).

Proprietà Funzionali dei Dati (`owl:FunctionalProperty`): Sono dichiarate le proprietà `numeroAnelli` (int), `haTrattoVerticale` (bool) e `haBasePesante` (bool). Il vincolo di funzionalità assicura che a ciascun individuo sia associato un unico valore per proprietà, garantendo la coerenza logica ed escludendo stati ambigui.

Assiomi di Definizione Concettuale:

$\text{CifraOtto} \equiv \text{CifraConAnelli} \sqcap (\geq 2 \text{ numeroAnelli})$

$\text{CifraQuattro} \equiv \text{CifraConAnelli} \sqcap (= 1 \text{ numeroAnelli}) \sqcap \text{haTrattoVerticale.true}$

$\text{CifraSei} \equiv \text{CifraConAnelli} \sqcap \text{haBasePesante.true} \sqcap (\text{haTrattoVerticale.true} \sqcup (\geq 1 \text{ numeroAnelli}))$

$\text{CifraZero} \equiv \text{CifraConAnelli} \sqcap (= 1 \text{ numeroAnelli}) \sqcap \text{haTrattoVerticale.false}$

$\text{CifraUno} \equiv \text{CifraTrattoSemplice} \sqcap (= 0 \text{ numeroAnelli}) \sqcap \text{haTrattoVerticale.true} \sqcap \text{haBasePesante.false}$

3.2.2 Popolamento Dinamico dell'ABox e Disambiguazione Congiunta

L'estensione non opera in maniera isolata, ma legge direttamente l'ABox contingente generata da `main.py`:

Caricamento dei Fatti: Individua l'ultimo file `kb/fatti_n_X.txt`, estrae la matrice 8x8 dei pixel e rileva la classe candidata dedotta dall'Albero.

Estrazione Topologica: Analizza la presenza di cavità/anelli chiusi (analisi del vuoto contornato da pixel attivi), continuità di colonne verticali e asimmetria nella distribuzione del peso visivo tra la metà superiore e inferiore.

Istanziamento ed Inferenza: Genera un individuo nominato (es. campione_cifra_6), ne asserisce i valori misurati per le proprietà funzionali ed esegue la classificazione deduttiva tra le classi della TBox.

Esito Congiunto: Se la classe inferita dagli assiomi coincide con la predizione dell'Albero, viene emessa una conferma formale di coerenza semantica; in caso di discrepanza morfologica viene segnalata una contraddizione esplicita; in assenza di assiomi sufficienti vige l'Open World Assumption (OWA).

3.2.3 Persistenza degli Artefatti Semantici (TBox vs ABox)

Al fine di rendere più semplice la lettura e interpretazione, il sistema preserva la netta separazione concettuale salvando due distinti file in standard W3C RDF/XML:

kb/digits_bk.owl: Contiene la TBox pura, ovvero la Background Knowledge astratta e invariante del dominio.

kb/ontologia_n_X.owl: Contiene la TBox integrata con l'ABox specifica dell'istanza esaminata a run-time (l'individuo, i suoi tratti misurati e i concetti inferiti), garantendo la tracciabilità ontologica della decisione.

4.0 Apprendimento Supervisionato (Classificazione , Modelli di base e valutazione)

L'apprendimento supervisionato è la capacità di un agente di migliorare il suo comportamento tramite l'esperienza passata, migliorandone l'accuratezza e dando all'agente la possibilità di fare più cose. E' un obiettivo di apprendimento automatico caratterizzato da un insieme di features (di input e di target) dove l'obiettivo è predire i valori per le feature target per gli esempi di test non ancora visti.

La base di conoscenza è rappresentata dal dataset standard digits di Scikit-learn, suddiviso in feature (data, array di 64 elementi) e target (target, valori da 0 a 9).

I modelli di ragionamento e apprendimento adottati per risolvere il problema di classificazione multiclasse sono l'Albero di Decisione, la Support Vector Machine (SVC) e il K-Nearest Neighbors (KNN).

La complessità delle regole per l'Albero di Decisione è limitata a una profondità massima pari a (`max_depth=20`), riducendo il rischio di overfitting e limitando la complessità del ragionamento a un massimo di 20 controlli sequenziali sui pixel

La Formalizzazione del Task e lo Spazio delle Feature

La classificazione è multiclasse e richiede che il sistema partizioni lo spazio vettoriale a 64 dimensioni in regioni con confini netti, assegnando ciascuna regione a una specifica cifra. Ogni pixel della griglia funge da feature continua normalizzata in scala di grigi: l'efficacia del classificatore dipende strettamente dalla sua capacità di catturare le correlazioni spaziali e la continuità dei tratti calligrafici distribuiti lungo la matrice.

5.0 Metodologia: Validazione e Prevenzione dell'Overfitting

L'overfitting è il fenomeno in cui un modello si adatta al rumore del training set, perdendo capacità di generalizzazione (basso bias, altissima varianza). Per mitigare il problema e stimare le performance in modo non distorto, il progetto implementa una Nested Cross-Validation.

Per garantire che le stime siano imparziali, ho adottato la tecnica della **Nested Cross-Validation** (validazione incrociata nidificata), strutturata su due cicli concentrici.

Livello	Configurazione	Funzione Operativa
Outer CV	5-Fold Cross Validation	Suddivide l'intero dataset in 5 blocchi bilanciati. A rotazione, 4 blocchi (80%) costituiscono il training set esterno e il restante blocco (20%) funge da test set cieco per la stima dell'errore di generalizzazione.
Inner CV	3-Fold Cross Validation	Eseguita internamente su ciascuna porzione dell'80% tramite GridSearchCV. Esplora le combinazioni di parametri addestrando sui due terzi e validando sul terzo rimanente, selezionando i best parameters senza toccare il test esterno.

Questo disaccoppiamento ci ha permesso di prevenire il data leakage.

Il **data leakage** (o "fuga di dati") si verifica quando informazioni provenienti dal set di test o di validazione contaminano in modo improprio il set di addestramento durante la fase di apprendimento di un modello.

STRUMENTI USATI

Libreria scikit-learn (sklearn): Rappresenta il framework fondamentale dell'intero progetto, impiegato per la gestione dei dataset, la modellazione predittiva, l'ottimizzazione degli iperparametri e la validazione statistica:

- `load_digits`: Modulo nativo per il caricamento del dataset benchmark di cifre manoscritte a 64 feature.
- `DecisionTreeClassifier`: Classe per la creazione, l'addestramento e la generazione dell'Albero di Decisione basato su criteri di impurità.
- `SVC` (Support Vector Classifier): Modulo per l'implementazione della Support Vector Machine ottimizzata per la classificazione multiclasse.
- `KNeighborsClassifier`: Classe dedicata all'apprendimento basato su istanze e *Case-Based Reasoning* tramite il calcolo delle distanze metriche.
- `GridSearchCV`: Strumento di ricerca esaustiva su griglia per l'individuazione automatica dei migliori iperparametri tramite validazione incrociata interna.
- `cross_validate` e `cross_val_predict`: Funzioni avanzate per l'esecuzione della valutazione incrociata esterna e per il calcolo dei punteggi di probabilità e funzione decisionale necessari alla generazione delle curve di performance.
- `metrics` (`precision_recall_curve`, `auc`, `label_binarize`): Moduli impiegati per la binarizzazione delle etichette (One-vs-Rest) e il calcolo delle metriche di precisione, richiamo e area sottesa alla curva (AUC).
- Libreria `owlready2`: Framework impiegato per la modellazione, manipolazione e serializzazione di ontologie OWL 2 in formato standard RDF/XML. Consente la formalizzazione dichiarativa della TBox, il vincolo funzionale delle proprietà, l'istanziamento dinamico dell'ABox e il ragionamento deduttivo direttamente in Python.
- Modulo `subprocess`: Modulo di sistema integrato in `main.py` per coordinare l'esecuzione interattiva a cascata del modulo estensione_OWL.py a valle della pipeline di machine learning.

Libreria pylab (matplotlib environment): Utilizzata per la gestione e il rendering grafico interattivo della curva Precision-Recall (Micro-Averaged), attivando la modalità di disegno in tempo reale (`pl.ion()`) per consentire la visualizzazione dei risultati senza bloccare l'esecuzione dello script principale.

Libreria numpy: Supporto matematico fondamentale per la manipolazione efficiente degli array multidimensionali, il calcolo vettoriale delle predizioni e la determinazione statistica di medie e deviazioni standard (`np.mean`, `np.std`) associate ai report delle metriche sui fold di validazione.

Decisioni di Progetto

Per evitare il bias di campionamento, prevenire il *data leakage* e garantire che le prestazioni e affidabilità delle valutazioni dei modelli da scelte arbitrarie, le predizioni vengono calcolate tramite la tecnica Nested Cross Validation con Outer CV a 5-Fold (per la stima dell' errore) e Inner CV a 3-Fold(per l' ottimizzazione) su tutti i modelli, al fine di garantire dei test su un modello addestrato sull' 80% dei dati.

Nelle fasi di validazione incrociata (cross_validate), ho configurato esplicitamente un dizionario di scoring basato su metriche macro-averaged (precision_macro, recall_macro, f1_macro). Questa scelta progettuale è fondamentale nei problemi di classificazione multiclasse poiché attribuisce lo stesso peso a ciascuna delle 10 cifre (da 0 a 9), impedendo che classi numeriche più semplici o più frequenti falsificano la valutazione globale delle performance (come invece accadeva in precedenza)

I 3 modelli sono configurati tramite i best parameters dati calcolati dalla GridSearchCV come segue:

Support Vector Machine (SVC): Tramite l'ottimizzazione ho potuto confermare l'impiego di un kernel='linear' con parametro di penalizzazione C=1. La scelta di un kernel lineare si fonda sulla natura dello spazio delle feature a 64 dimensioni: trattandosi di un dominio già sufficientemente ampio e continuo, la separazione lineare al margine massimo risulta estremamente efficace, evitando l'overhead computazionale e i rischi di overfitting legati a kernel non lineari più complessi

K-Nearest Neighbors (KNN): L'analisi dei vicini ha identificato in n_neighbors=3 il valore ottimale. Scegliere un valore di k ridotto permette di catturare le variazioni locali e morfologiche tipiche della scrittura manuale dei numeri, evitando l'effetto di appiattimento (alto bias) che si registrerebbe ampliando eccessivamente il raggio di vicinato.

Albero di Decisione (Decision Tree): Per ridurre la tendenza naturale del modello ad andare in overfitting memorizzando il training set, la complessità è stata vincolata impostando una profondità massima max_depth=20 e uno split minimo min_samples_split=2. Questo bilancia la necessità di disporre di un numero

sufficiente di livelli logici per discriminare le cifre con la necessità di arrestare la crescita dell'albero prima che intercetti il rumore casuale dei pixel.

Alberi di Decisione (Decision Tree)

Sommario e Fondamenti Teorici

L'Albero di Decisione è un modello simbolico interpretabile in cui ogni nodo interno esegue un test su una caratteristica, ogni ramo uscente corrisponde a un esito del test e ciascuna foglia assegna una classe. La costruzione dell'albero avviene con un algoritmo ricorsivo top-down che seleziona greedy la condizione che minimizza l'impurità dei nodi figli.

Strumenti Utilizzati

Ho stanziato il modello mediante DecisionTreeClassifier del modulo sklearn.tree. L'ottimizzazione è gestita da GridSearchCV.

Decisioni di Progetto e Spazio degli Iperparametri

Gli alberi non vincolati soffrono di un bias di rappresentazione quasi nullo ma di una varianza elevata, arrivando a memorizzare ogni singolo esempio del dataset. Per controllare la complessità e impedire che l'albero scenda in partizioni prive di significatività statistica, ho definito la seguente griglia:

```
param_grid_dt = {'max_depth': [None, 10, 20], 'min_samples_split': [2, 5]}
```

I valori esplorano la possibilità di lasciare l'albero libero (None) o di fissare tetti di profondità a 10 o 20 livelli, combinati con la richiesta di almeno 2 o 5 campioni per tentare una diramazione. La ricerca su griglia ha isolato come configurazione migliore `max_depth=20` e `min_samples_split=2`, confermando la necessità di profondità elevate per discriminare le cifre, pur mantenendo un vincolo rigido oltre il ventesimo livello decisionale. Il vincolo sulla profondità massima agisce direttamente sul compromesso tra bias e varianza: si scongiura l'altissima varianza tipica di un albero lasciato libero di memorizzare il rumore del dataset (fenomeno di overfitting), incrementando lievemente il bias costruttivo. Questa operazione di regolarizzazione della complessità del modello, dettata dal principio del Rasoio di Occam e dalla Minimum Description Length (MDL), ottimizza la capacità di generalizzazione predittiva su nuovi campioni.

Valutazione Sperimentale

I report estratti dalle 5 iterazioni di validazione esterna hanno restituito i seguenti punteggi medi e deviazioni standard:

Metrica (Nested 5-Fold CV)	Valore Medio	Deviazione Standard ($\pm \sigma$)
Macro Precision	0.802	± 0.037
Macro Recall	0.794	± 0.043
Macro F1-Score	0.791	± 0.045
Area Under Curve (AUC)	0.794	-

Support Vector Machine (SVC)

Sommario e Fondamenti Teorici

La Support Vector Machine estende i modelli lineari ricercando l'iperpiano di separazione che massimizza il margine geometrico tra le classi. Gli elementi del dataset che si trovano esattamente sul confine del margine prendono il nome di vettori di supporto. La massimizzazione del margine si formula analiticamente come un problema di ottimizzazione convessa quadratica vincolata, garantendo l'assenza di minimi locali e conducendo a un unico ottimo globale.

Strumenti Utilizzati

Per questo modello mi sono affidato alla classe SVC del modulo `sklearn.svm`, gestendo la classificazione multiclasse con architettura One-vs-One (ovvero combinando 45 classificatori binari a coppie).

Decisioni di Progetto e Spazio degli Iperparametri

Nello spazio delle feature a 64 dimensioni, la rappresentazione è già sufficientemente sparsa da consentire un'eccellente separazione lineare diretta.

Pertanto, lo spazio di ricerca è stato strutturato definendo:

```
param_grid_svc = {'C': [1, 10, 100], 'kernel': ['linear']}
```

Il parametro di regolarizzazione C agisce come costante di penalità per le violazioni del margine (soft margin): valori più alti impongono margini rigidi penalizzando gli errori, mentre valori moderati consentono una migliore tolleranza alle ambiguità calligrafiche. L'ottimizzazione tramite Inner CV ha confermato come parametro migliore $C=1$ con `kernel='linear'`.

Valutazione Sperimentale

I risultati della validazione nidificata esterna mostrano un netto miglioramento delle prestazioni infatti:

Metrica (Nested 5-Fold CV)	Valore Medio	Deviazione Standard ($\pm \sigma$)
Macro Precision	0.950	± 0.021
Macro Recall	0.948	± 0.021
Macro F1-Score	0.947	± 0.021
Area Under Curve (AUC)	0.961	-

Case-Based Reasoning (K-Nearest Neighbors)

Sommario e Fondamenti Teorici

L'algoritmo K-Nearest Neighbors implementa il paradigma del Case-Based Reasoning (CBR) e dell'apprendimento basato su istanze (instance-based learning). A differenza degli altri approcci che costruiscono un modello compatto offline, il KNN non generalizza nella fase di training: memorizza i casi noti e rimanda l'intero sforzo computazionale alla fase online. Quando giunge una nuova istanza, ne calcola la distanza euclidea rispetto a tutti i punti della base di conoscenza e attribuisce la classe più frequente (moda) tra i k vicini più prossimi.

Strumenti Utilizzati

Il modello adotta la classe `KNeighborsClassifier` del modulo `sklearn.neighbors`, affiancata da `GridSearchCV` per l'ottimizzazione del raggio di vicinato.

Decisioni di Progetto e Spazio degli Iperparametri

Il valore di k governa direttamente la complessità del modello: un k troppo piccolo (es. 1) genera superfici decisionali frammentate e vulnerabili al rumore (alta varianza), mentre un k eccessivamente alto appiattisce le peculiarità locali a favore delle classi maggioritarie (alto bias). La griglia impostata esplora valori dispari per prevenire situazioni di parità:

```
param_grid_knn = {'n_neighbors': [3, 5, 7]}
```

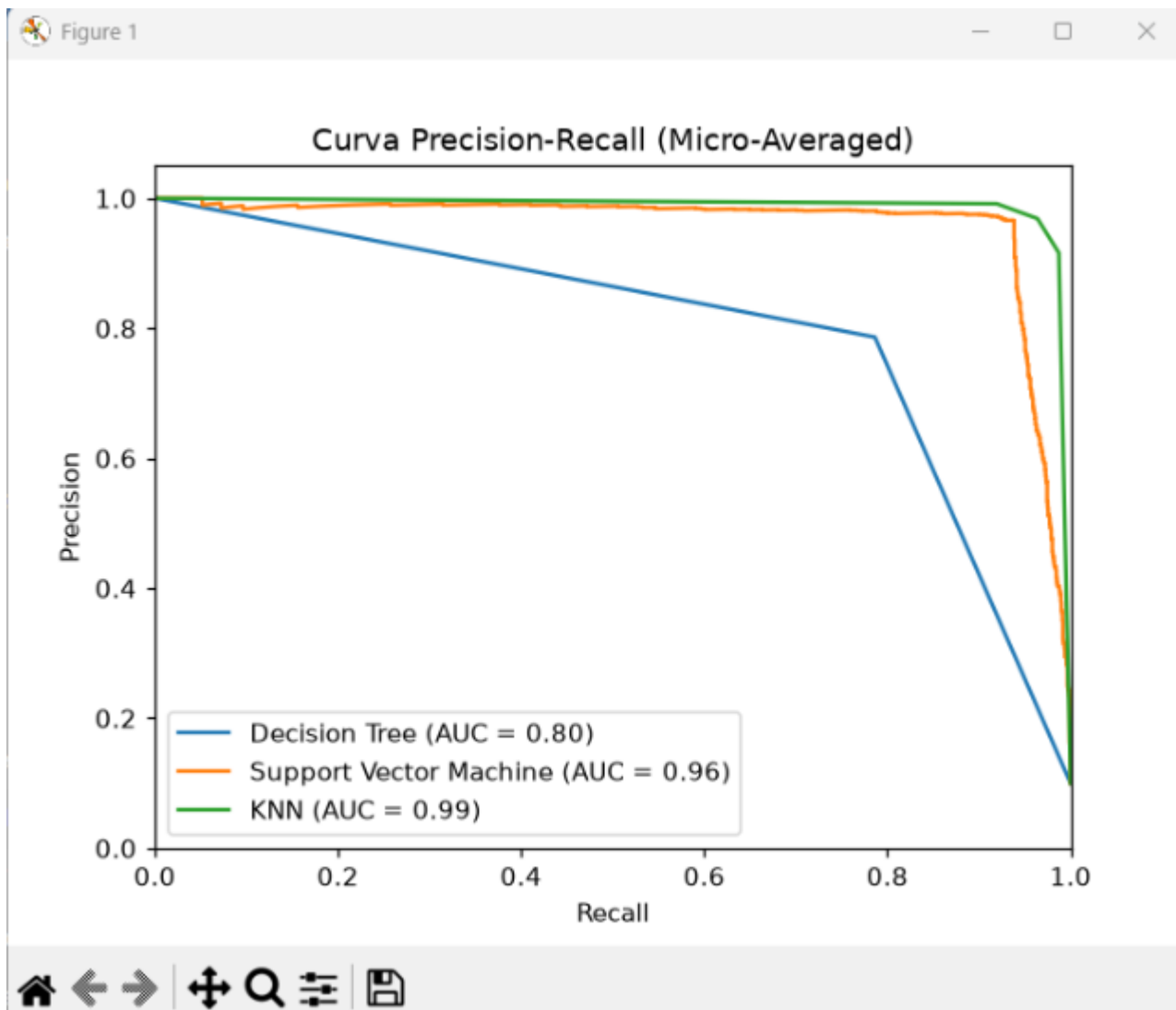
La validazione interna mi ha confermato $n_neighbors=3$ come configurazione ottimale, capace di catturare le variazioni locali del tratto numerico senza farsi condizionare da punti isolati.

Valutazione Sperimentale

Il KNN ha ottenuto i valori di accuratezza e separazione più elevati dell'intero studio:

Metrica (Nested 5-Fold CV)	Valore Medio	Deviazione Standard ($\pm \sigma$)
Macro Precision	0.967	± 0.008
Macro Recall	0.965	± 0.009
Macro F1-Score	0.965	± 0.009
Area Under Curve (AUC)	0.988	-

VALUTAZIONE DEI RISULTATI



Il grafico mostra l'andamento tra la **Precision** (asse y: l'accuratezza delle predizioni positive) e la **Recall** (asse x: la capacità del modello di catturare tutte le istanze positive) per ciascun classificatore.

- Più la curva si sposta verso l'angolo in alto a destra, migliori sono le performance del modello.
- L'indicatore sintetico **AUC (Area Under Curve)** riassume l'efficacia complessiva: il *Decision Tree* ottiene un valore di 0.80, la *Support Vector Machine* raggiunge 0.96, mentre il *KNN* domina con un valore quasi perfetto di 0.99. Questo conferma visivamente che KNN e SVC mantengono una precisione elevatissima anche a livelli di richiamo molto alti, superando nettamente l'albero di decisione.

Valutazione della Matrice di Confusione:

Ad integrazione dei report globali e delle curve PR, nel progetto è stata inclusa la stampa esplicita a console della Matrice di Confusione multiclasse (10x10) calcolata mediante `cross_val_predict` per l'Albero di Decisione. Analizza puntualmente la distribuzione degli errori di I tipo (falsi positivi) e di II tipo (falsi negativi) su ciascuna cifra. L'ispezione della matrice ha mostrato come la maggior parte degli errori dell'albero si concentri su cifre con forti sovrapposizioni morfologiche e simmetrie calligrafiche (ad esempio lo scambio tra la classe '3' e la classe '8', o tra '1' e '7'), fornendo una spiegazione quantitativa del minor valore di AUC e F1-Score registrato dall'Albero rispetto a modelli geometrici come SVC e KNN.

```
=====
MATRICE DI CONFUSIONE - Albero di Decisione
=====
Righe: Cifra Reale (0-9) | Colonne: Cifra Predetta (0-9)
```

[	164	0	2	1	5	2	2	0	2	0]
[	0	135	12	7	1	5	2	1	9	10]
[	1	14	124	8	1	1	4	7	17	0]
[	0	4	8	132	2	5	2	4	11	15]
[	1	16	1	3	133	2	11	6	4	4]
[	8	1	0	2	11	143	4	2	2	9]
[	1	4	1	0	4	2	164	3	2	0]
[	2	3	1	5	10	2	1	147	5	3]
[	3	13	5	9	3	6	1	0	128	6]
[	0	7	3	11	1	12	0	9	7	130]]

STAMPA DELL' ALBERO

Ho deciso di implementare una ricerca euristica A* per analizzare l'albero generato dall'addestramento. Questo ha permesso di evitare l'esplorazione cieca sull'albero usando una coda di priorità per decidere quale sarà il nodo da esplorare per primo.

Sfrutta l'euristica $h(n) + \text{cost}(n)$ (il costo effettivo del percorso parziale), che permette di trovare il cammino a costo minimo.

Per tanto ho implementato una stampa a video del target, il cammino fatto dall'algoritmo tramite A*, dove 0 è il nodo radice e i valori numerici all'interno della parentesi è il nodo.

Nella seguente immagine si nota che sono stati necessari 5 test prima di ottenere la risposta ottimale.

```
=====
RICERCA A* NELL'ALBERO DI DECISIONE
=====
Classe target cercata: 4
Percorso nodi (radice -> foglia): [0, np.int64(1),
np.int64(2), np.int64(3), np.int64(5), np.int64(11)]
Costo del percorso (profondità): 5
```

STAMPA DEL' OUTPUT

Oltre alla stampa delle tabelle precedentemente mostrate, il programma stampa i valori predetti per verificare che i 3 metodi funzionano.

```
IL VERO NUMERO DA PREDIRE E': 8

=====
NUMERO PREDETTO
=====
Albero decisione: 8
                  SVC: 8
                  Knn: 8

[OK] Tutte le regole salvate in: 'kb\regole_n_8.txt'
[OK] Fatti osservati del campione salvati in: 'kb\fatti_n_8.txt'
```

A questa fase empirica è stata affiancata la derivazione della Knowledge Base: il sistema genera nella directory /kb il file con tutte le regole logiche IF-THEN apprese (regole_n_{target}.txt) e il file con gli assiomi ground (fatti_osservati.txt) relativi al campione estratto a run-time. Contestualmente, la funzione inferenza_kb valuta tali fatti e stampa a console l'esatta catena deduttiva attivata.

OUTPUT ESEMPIO

Cifra dedotta formalmente: 8

Regola attivata: #115

Catena logica: IF (pixel_36 > 0.50) AND (pixel_21 > 0.50) AND (pixel_60 > 7.50) AND (pixel_33 <= 3.50) AND (pixel_43 > 1.50) AND (pixel_27 > 7.50) AND (pixel_20 <= 15.50) AND (pixel_18 > 2.50) AND (pixel_38 <= 3.50) AND (pixel_21 > 4.00) THEN Cifra = 8

AVVIO DELLA ESTENSIONE ONTOLOGICA-

```
Vuoi avviare l'estensione ontologica? (Y/N): y
Caricamento fatti dall'istanza registrata in: kb\fatti_n_8.txt
Ground Truth reale: 8 | Predizione Albero registrata: 8

=====
RAGIONAMENTO ONTOLOGICO FORMALE (OWL 2)
=====

Fatti estratti dall'ABox:
- Individuo asserito:      campione_cifra_8
- Numero anelli chiusi rilevati: 2
- Presenza tratto verticale: True
- Base con densita' maggiore: False

Concetti inferiti per il campione: ['Cifra', 'CifraOtto']

-----
DISAMBIGUAZIONE CONGIUNTA (DECISION TREE + ONTO-BK)
-----

Predizione candidata derivata da main.py: Cifra 8
Esito: L'ontologia CONFERMA formalmente la classe 8 per coerenza assiomatica.

[OK] Ontologia del campione salvata in: 'kb\ontologia_n_8.owl'
[OK] Ontologia di riferimento aggiornata in: 'kb\digits_bk.owl'
PS C:\Users\mloco\Desktop\3o anno\Ingegneria della Conoscenza\progetto\src>
```

L'algoritmo di analisi delle componenti connesse e dei vuoti racchiusi ha rilevato con successo la presenza di 2 cavità chiuse ($\text{numeroAnelli} = 2$). Tale evidenza costituisce la proprietà geometrico-topologica cardine della cifra 8.

Applicando gli assiomi della TBox

$(\text{CifraOtto} \equiv \text{CifraConAnelli} \sqcap (\geq 2 \text{ numeroAnelli}))$

Infine ha convalidato la predizione induttiva dell'Albero di Decisione, certificando che la classificazione numerica non è solo un pattern statistico ma rispetta rigorosamente la semantica a priori del dominio.

Conclusioni

Il progetto mi ha permesso di confrontare 3 modelli, che usano lo stesso dataset per identificare un numero senza l'uso di Computer Vision o Deep learning ma trattando il problema come una classificazione su vettori.

Dai dati ricevuti dai vari test si nota come KNN e SVC abbiano prestazioni migliori rispetto agli alberi di decisioni e infatti restituiscono previsioni più affidabili.

Inizialmente era stata implementata una divisione degli esempi più semplice (2 Fold Cross Validation) che non permetteva un buon addestramento dei modelli. Questo è stato risolto tramite la Outer Fold a 5 e Inner Fold a 3. Quindi ho reputato essenziale fare stampare sempre il numero da predire e le reali predizioni. Nelle svariate prove tutti e tre i metodi sono riusciti a predire il numero, facendomi escludere la possibilità di dover usare Leave-One-Out Cross Validation per migliorare le predizioni.

Tramite la Nested Cross validation ho potuto risolvere il problema dell'overfitting dimostrando che i modelli hanno capacità di generalizzare.

ESTENSIONE

L'integrazione del modulo ontologico OWL 2 ha elevato il sistema a un'architettura neuro-simbolica ibrida (ML + OntoBK): le decisioni derivate dai dati numerici empirici vengono vincolate e validate a fronte di assiomi topologici formali a priori. La separazione fisica tra la TBox generale (digits_bk.owl) e le ABox contestualizzate sui singoli campioni (ontologia_n_X.owl) l'interoperabilità nel Web Semantico.

Riferimenti Bibliografici

[1] Scikit-learn Developers: *Machine Learning in Python*. Documentazione ufficiale dei moduli sklearn.svm, sklearn.tree, sklearn.neighbors e sklearn.metrics.

[2] N. Fanizzi, "Ingegneria della Conoscenza", Dispensa del Corso di Laurea in Informatica.

[3] W3C: OWL 2 Web Ontology Language Document Overview (Second Edition). Raccomandazione ufficiale W3C.